用 ruby 玩音乐

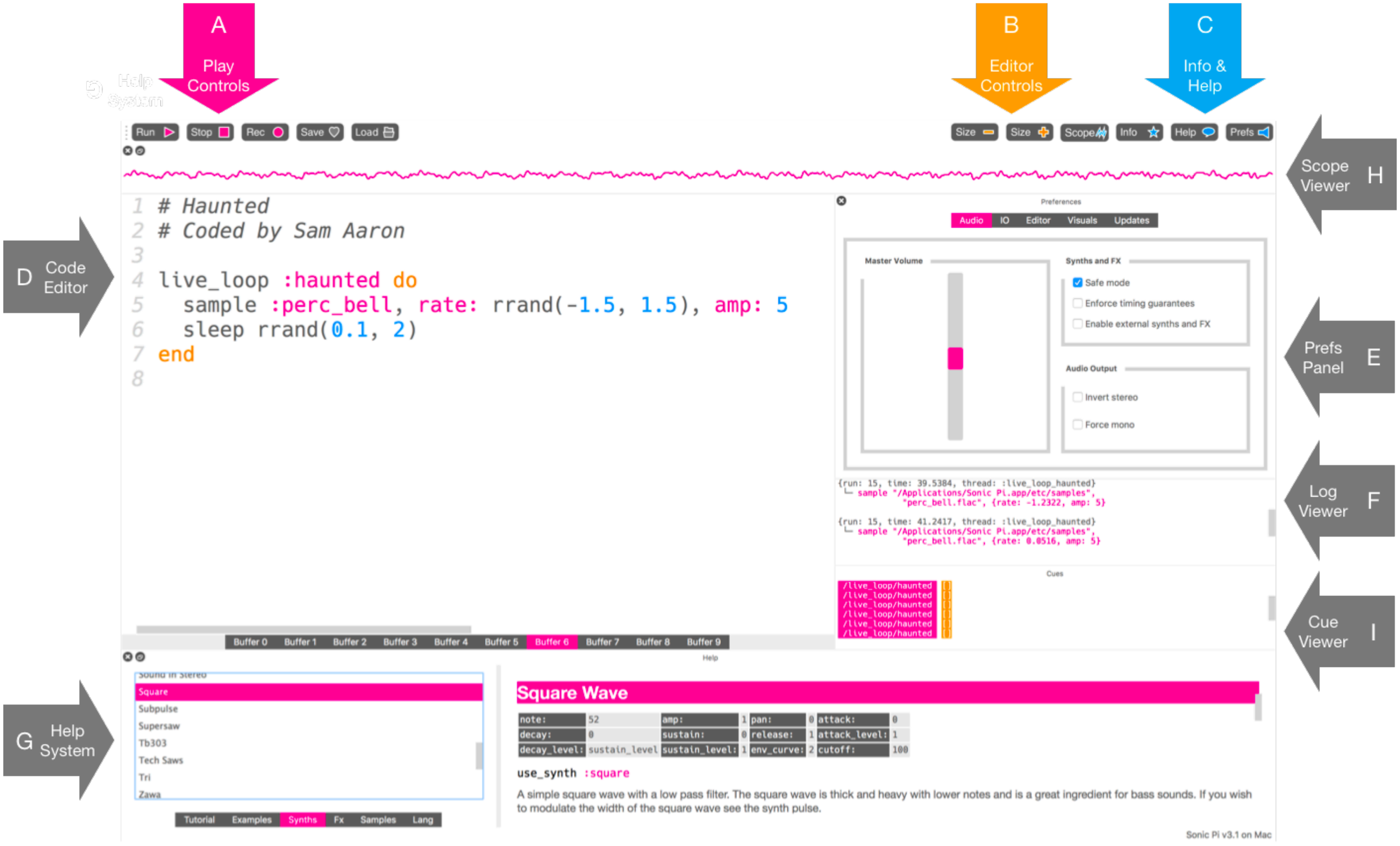
yong.zhang

Outline

- Sonic Pi 项目介绍
- Sonic Pi 实现原理浅析
- 几个有趣的实验
- 应用

Sonic Pi 项目介绍

- 一个开源的编程环境
- 一种乐器


A
Play Controls
B
Editor Controls
C
Info & Help
Run
Stop
Rec
Save
Load
Size
Size
Scope
Info
Help
Prefs
Scope Viewer H
D Code Editor
Haunted
Coded by Sam Aaron
live_loop :haunted do
sample :perc_bell, rate: rrand(-1.5, 1.5), amp: 5
sleep rrand(0.1, 2)
end
Preferences
Audio
IO
Editor
Visuals
Updates
Master Volume
Synths and FX
Safe mode
Enforce timing guarantees
Enable external synths and FX
Audio Output
Invert stereo
Force mono
Prefs Panel E
{run: 15, time: 39.5384, thread: :live_loop_haunted}
sample "/Applications/Sonic Pi.app/etc/samples", "perc_bell.flac", {rate: -1.2322, amp: 5}
{run: 15, time: 41.2417, thread: :live_loop_haunted}
sample "/Applications/Sonic Pi.app/etc/samples", "perc_bell.flac", {rate: 0.0516, amp: 5}
Log Viewer F
Cues
/live_loop/haunted
Cue Viewer I
Buffer 0
Buffer 1
Buffer 2
Buffer 3
Buffer 4
Buffer 5
Buffer 6
Buffer 7
Buffer 8
Buffer 9
Help
G Help System
Square
Subpulse
Supersaw
Tb303
Tech Saws
Tri
Zawa
Tutorial
Examples
Synths
Fx
Samples
Lang
Square Wave
note: 52
amp: 1
pan: 0
attack: 0
decay: 0
sustain: 0
release: 1
attack_level: 1
decay_level: sustain_level
sustain_level: 1
env_curve: 2
cutoff: 100
use_synth :square
A simple square wave with a low pass filter. The square wave is thick and heavy with lower notes and is a great ingredient for bass sounds. If you wish to modulate the width of the square wave see the synth pulse.
Sonic Pi v3.1 on Mac

Getting started

演奏音符

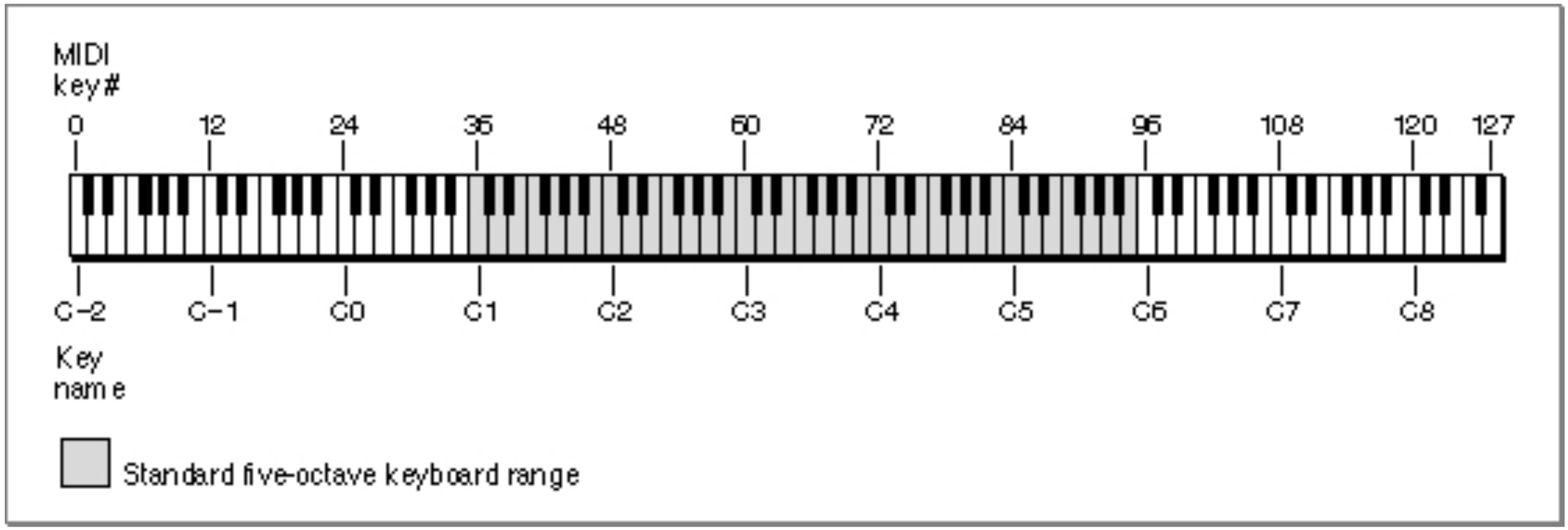
```
play 60
```

多个音符一起演奏

```
play 60
play 67
play 69
```

停顿

```
sleep 1
```



MIDI note values

Getting started

旋律

```
play 60
```

和声

```
play 60
play 67
play 69
```

节奏

```
sleep 1
```

Getting started

音符参数

```
play 60, attack: 1, release: 3
```

音色

```
use_synth :fm
2.times do
  play 60
  sleep 0.5
  play 67
  sleep 0.5
end
```

效果

```
with_fx :reverb do
  with_fx :distortion do
    sample :guit_e_fifths
  end
end
```

Getting started

多音轨

```
in_thread do
  loop do
    sample :loop_amen
    sleep 1.753
  end
end
```

超级马里奥

乐曲结构

```
use_bpm 100
use_synth :pulse
use_synth_defaults release: 0.2, mod_rate: 5, amp: 0.6
define :structure do |i,a,b,c,d|
  1.times { i.call }
  loop do
    2.times { a.call }
    2.times { b.call }
    1.times { c.call }
    2.times { a.call }
    2.times { d.call }
    1.times { c.call }
    1.times { d.call }
  end
end
...
...
```

Sonic Pi 原理浅析

音频合成原理

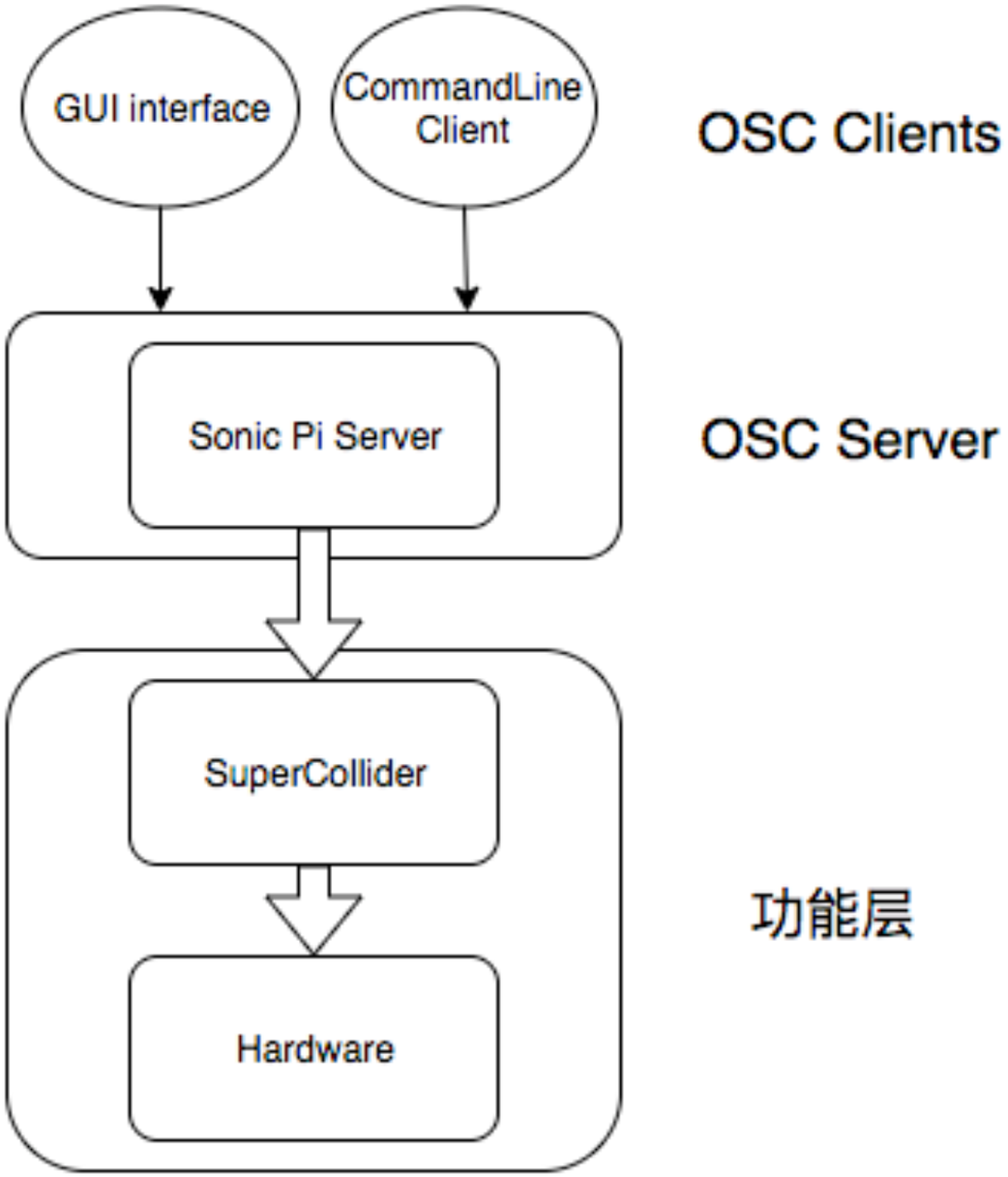
- 硬件：蜂鸣器、音频合成器

- Audio Server — SuperCollider

  - API http://doc.sccode.org/Reference/Server-Command-Reference.html

软件架构

- 协议: OSC (Open Sound Control)
- 服务端: 调用 SuperCollier API
- 客户端: GUI Client, Common OSC client



源码浅析~服务启动

```
begin
  if protocol == :tcp
    osc_server = SonicPi::OSC::TCPServer.new(server_port, use_decoder_cache: true)
  else
    osc_server = SonicPi::OSC::UDPServer.new(server_port, use_decoder_cache: true)
  end
rescue Exception => e
  begin
    STDOUT.puts "Exception when opening a socket to listen from GUI!"
    STDOUT.puts e.message
    STDOUT.puts e.backtrace.inspect
    STDOUT.puts e.backtrace
    gui.send("/exited-with-boot-error", "Failed to open server port " + server_port.to_s + ", is scsynth already running?")
  rescue Errno::EPIPE => e
    STDOUT.puts "GUI not listening, exit anyway."
  end
  exit
end
```

app/server/ruby/bin/sonic-pi-server.rb

源码浅析~API 接口

```ruby
osc_server.add_method("/run-code") do |args|
  gui_id = args[0]
  code = args[1].force_encoding("utf-8")
  sp.__spider_eval code
end

osc_server.add_method("/save-and-run-buffer") do |args|
  gui_id = args[0]
  buffer_id = args[1]
  code = args[2].force_encoding("utf-8")
  workspace = args[3]
  sp.__save_buffer(buffer_id, code)
  sp.__spider_eval code, {workspace: workspace}
end

osc_server.add_method("/save-buffer") do |args|
  gui_id = args[0]
  buffer_id = args[1]
  code = args[2].force_encoding("utf-8")
  sp.__save_buffer(buffer_id, code)
end
```

app/server/ruby/bin/sonic-pi-server.rb

源码浅析~功能实现

启动 SuperCollider audio server

```
@osc_server = OSC::UDPServer.new(0, use_decoder_cache: true, use_encoder_cache: true)
```

```
case os
when :raspberry
  boot_server_raspberry_pi
when :linux
  boot_server_linux
when :osx
  boot_server_osx
when :windows
  boot_server_windows
end
```

发送 API 请求

```
def send(*all_args)
  address, *args = *all_args
  log "OSC            ~ #{address} #{args.inspect}" if osc_debug_mode
  @osc_server.send(@hostname, @send_port, address, *args)
end
```

app/server/ruby/lib/sonicpi/scsynthexternal.rb

几个有趣的实验

实验1: 声音的和谐

- 为什么有的音一起听起来和谐，有的刺耳?
- 为什么是十二平均律?

实验2: 不同的音色

- 为什么听声音就可以辨别人?
- 为什么不同乐器演奏的相同音高会有区别?

音乐背后的理性

实验3: 旋律

- 随机的旋律
- 中国五声音阶 vs 日本五声音阶

实验4: 和声

- 加上伴奏 — 有厚度的音乐
- 大调音乐与小调音乐

音乐不是纯理性

LONIAN 大调式	1 2 3 4 5 6 7 1	
DORIAN	2 3 4 5 6 7 1 2	
PHRYGIAN	3 4 5 6 7 1 2 3	
LYDIAN	4 5 6 7 1 2 3 4	
MIXOLYDIAN	5 6 7 1 2 3 4 5	
AEOLIAN 小调式	6 7 1 2 3 4 5 6	
LOCRIAN	7 1 2 3 4 5 6 7	

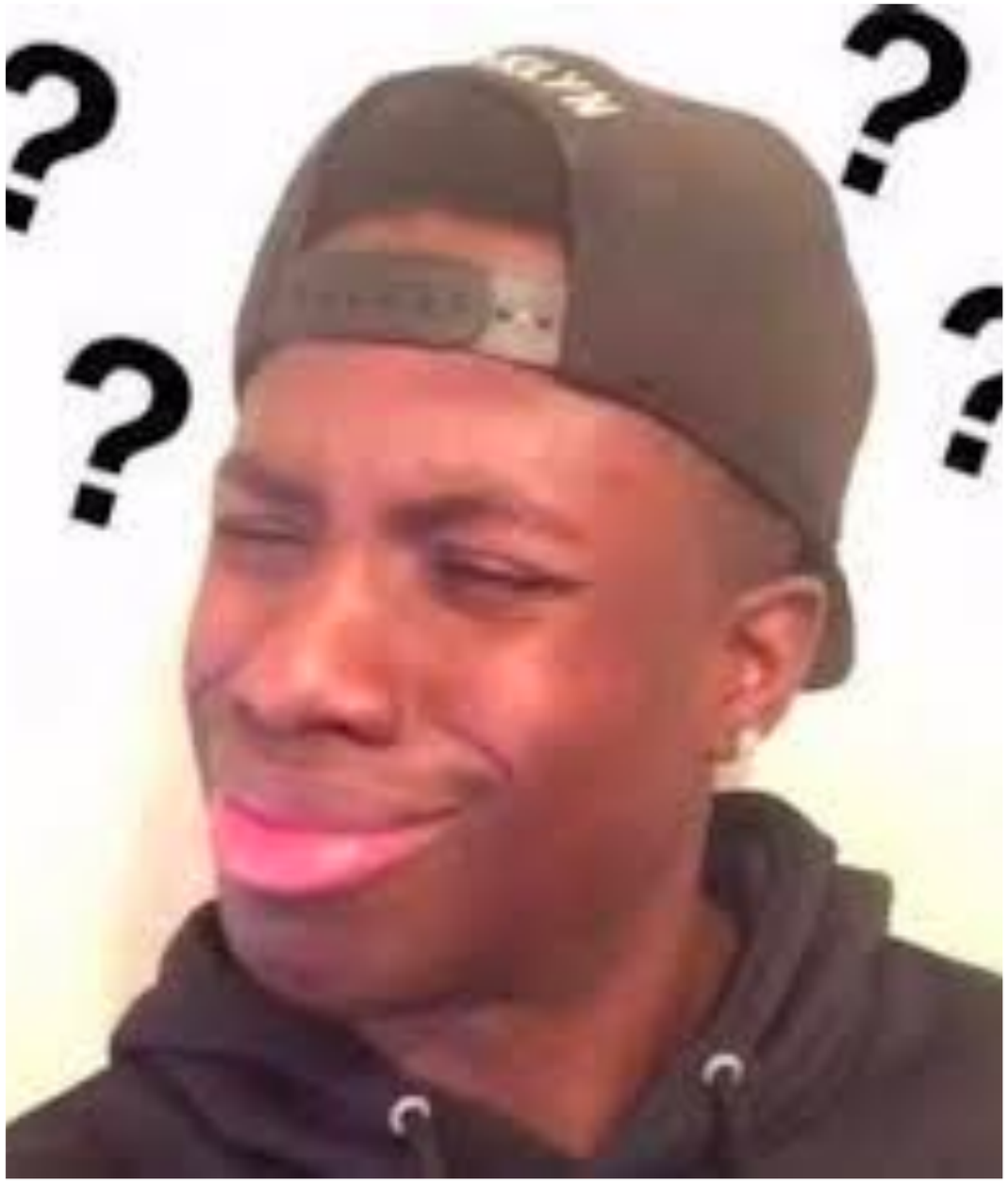
应用

一种新型乐器

- 优势
  - 上手简单，不用长期的训练
  - 更加精确的控制
  - 快速验证，方便保存、修改
  - 更方便地折腾，如算法谱曲
- 传统乐器是否会被取代?

编程教育

- 优势
  - 简单
  - 有趣



Having fun with sorting?

Thank you